

# Post-Quantum Token

*Lamport tek-kullanımlık imzalarla kuantum-dirençli token*

---

**Ahmet Murat Sencay**

Öğrenci No: 220304038



## ■ Problem: Neden Post-Kuantum?

- Bugünkü blok zincirlerde işlemler ECDSA (eliptik eğri) imzasıyla yetkilendirilir.
- Yeterince güçlü bir kuantum bilgisayar + Shor algoritması, açık anahtardan gizli anahtarı geri çıkarabilir.
- Bu, ECDSA ve RSA'yı tamamen kırar — saldırgan herkesin imzasını taklit edip parasını harcayabilir.
- İhtiyaç: yalnızca güvenli varsayımlara (hash fonksiyonu) dayanan, kuantuma dirençli imzalar.



**ECDSA / RSA**

**kuantum bilgisayarda KIRILIR**

Shor algoritması gizli anahtarı  
açık anahtardan polinom zamanda bulur

## ■ Shor vs Grover: Neden Hash Tabanlı?



### Shor Algoritması

- Çarpanlara ayırma ve ayrık logaritmayı polinom zamanda çözer.
- RSA, Diffie-Hellman, ECDSA → tamamen KIRILIR.
- Açık anahtardan gizli anahtar elde edilir.



### Grover Algoritması

- Arama problemlerinde yalnızca karekök hızlanma sağlar.
- Hash güvenliğini yarıya indirir: 256-bit → ~128-bit.
- Hash'i KIRMAZ, sadece zayıflatır → 256-bit hâlâ güvenli.



**Sonuç:** Hash tabanlı imzalar (Lamport) yalnızca bir hash fonksiyonuna dayandığı için post-kuantum güvenli kabul edilir.

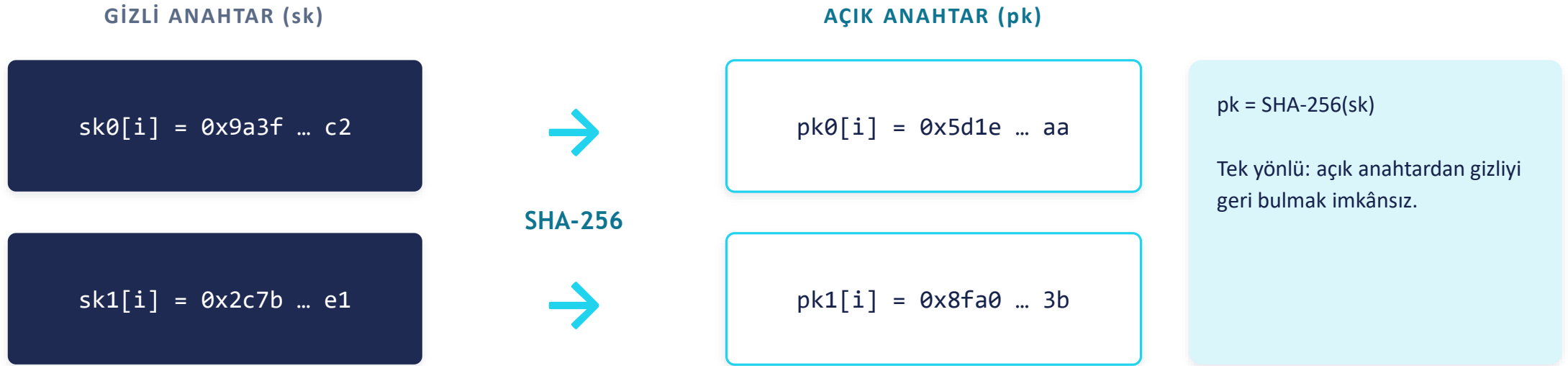
## ■ Lamport İmzası: Temel Fikir



**Sadece bir hash fonksiyonuna (SHA-256) dayanır — sayı teorisi yok.**

Her anahtar yalnızca BİR kez güvenle kullanılır → “tek kullanımlık” (one-time signature). İmzaladıktan sonra yeni anahtara geçilir.

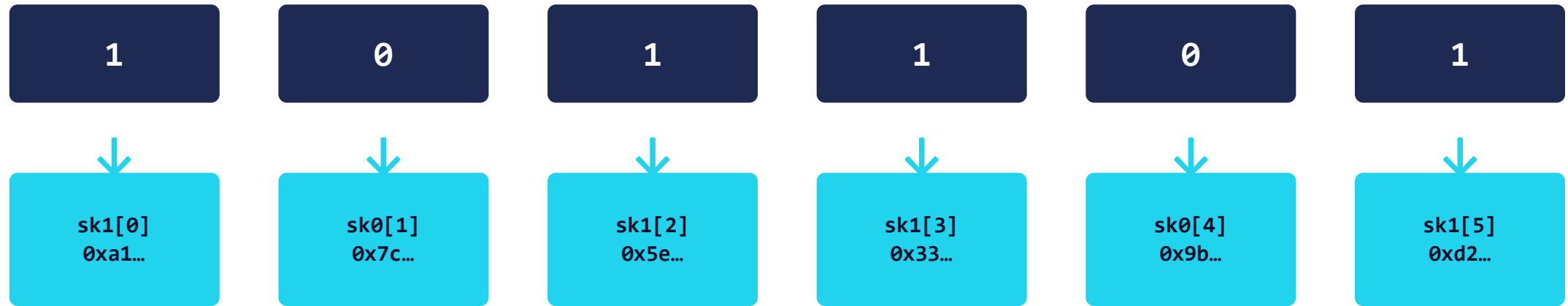
# ■ 1) Anahtar Üretimi



- Her bit için İKİ gizli değer üretilir:  $sk0[i]$  ve  $sk1[i]$ .
- Açık anahtar, her gizli değerın SHA-256 hash'idir.
- $N = 256$  bit  $\rightarrow$  512 gizli değer ve 512 açık değer.

## ■ 2) İmzalama

Mesajın SHA-256 hash'i 256 bit. Her bit, hangi gizli değerin açığa çıkacağını belirler:



*seçilen gizli değerler = imza*

**Bit 1 ise sk1[i], bit 0 ise sk0[i] açığa çıkar. İmza = 256 seçilmiş gizli değer. Her bit için yalnızca BİR yarı açılır; diğer yarı gizli kalır.**

### ■ 3) Doğrulama

Her bit için aynı kontrol yapılır:



256 bitin HEPSİ eşleşirse → imza GEÇERLİ (true)



Biri bile tutmazsa → GEÇERSİZ (false)

*Bu, kontrattaki verifyLamport fonksiyonunun tam olarak yaptığı iştir.*

## ■ Neden Tek-Kullanımlılık? + Anahtar Rotasyonu



### Tehlike: ikinci imza

- Bir imza, her bit için gizli değerin yalnızca yarısını açığa çıkarır.
- Aynı anahtarla ikinci kez imzalarsan farklı bitler için daha çok gizli değer sızar.
- Saldırgan sızan parçaları birleştirip kendi mesajı için SAHTE imza üretebilir.

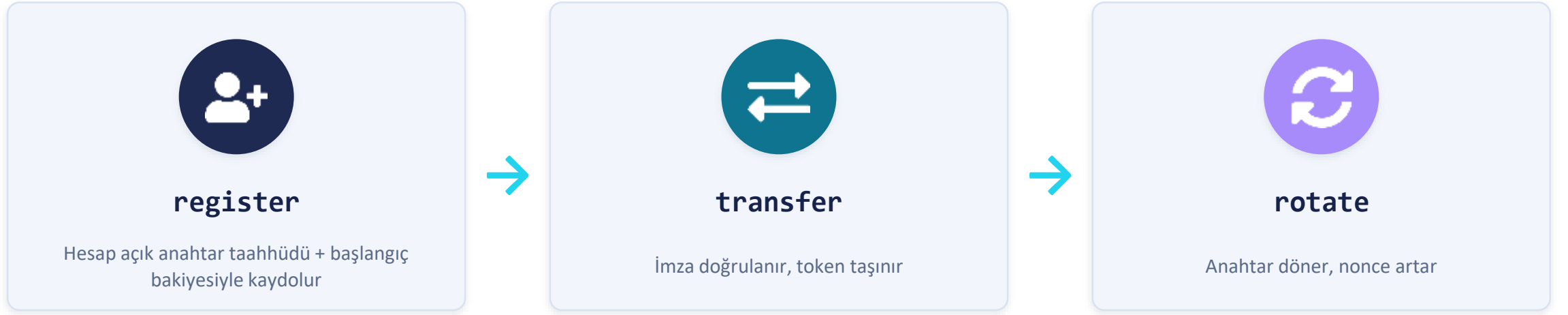


### Çözüm: anahtar rotasyonu

- Her imzadan (transferden) sonra YENİ bir anahtara geçilir.
- Token'da: her transfer, bir sonraki açık-anahtar taahhüdünü (nextCommitment) belirtir.
- Eski anahtar bir daha asla kullanılmaz → güvenlik korunur.



**ERC-20 benzeri bir token** — ama transferleri ECDSA değil, **Lamport imzası** yetkilendirir.



- Her hesap, açık anahtarının kısa bir TAAHHÜDÜNÜ (commitment) saklar — tüm anahtarı değil.
- Transfer akışı: taahhüt kontrolü → imza doğrulama → bakiye → token taşıma → anahtar rotasyonu.

## ■ verifyLamport — Doğrulamanın Kalbi

```
function verifyLamport(  
    bytes32 msgHash,  
    bytes32[256] calldata signature,  
    bytes32[256] calldata pk0,  
    bytes32[256] calldata pk1  
) public pure returns (bool) {  
    for (uint256 i = 0; i < N; i++) {  
        uint256 bit = (uint256(msgHash) >> i) & 1;  
        bytes32 expected = bit == 0 ? pk0[i] : pk1[i];  
        if (sha256(abi.encodePacked(signature[i]))  
            != expected) {  
            return false;  
        }  
    }  
    return true;  
}
```

### 1 Biti çek

msgHash'in i. bitini al → 0 ya da 1. (>> i ile kaydır, & 1 ile son bit oku)

### 2 Yarıyı seç

bit 0 ise pk0[i], bit 1 ise pk1[i] beklenir — doğru açık-anahtar yarısı.

### 3 Karşılaştır

imzadaki değeri sha256'la; beklenenle tutmazsa hemen false. 256 bit de tutarsa → true.

## ■ transfer: Güvenlik Sırası

1

**Geçerli alıcı** `to != address(0)`

*sıfır adrese (kayıp) transferi engeller*

2

**Doğru anahtar** `keccak256(pk0, pk1) == publicKeyCommitment[gönderen]`

*yalnızca gerçek anahtar sahibi harcayabilir — sahteciliği önler*

3

**Mesajı bağla** `msgHash = sha256(gönderen, to, amount, nonce, yeniTaahhüt)`

*imza bu transfere özgü olur — başkasına kopyalanamaz*

4

**İmzayı doğrula** `require( verifyLamport(msgHash, signature, pk0, pk1) )`

*256-bit Lamport doğrulaması*

5

**Yeterli bakiye** `require( balanceOf[gönderen] >= amount )`

*bakiyeden fazlası gönderilemez*

6

**Uygula** `taşı · taahhüdü döndür (rotate) · nonce++ · Transfer yay`

*eski anahtar bir daha kullanılamaz → replay engellenir*

## ■ Replay (Tekrar) Saldırısına Karşı Koruma



### Nonce sayacı

Her transferde artar → mesaj hash'i değişir → eski imza yeni transferde geçersiz olur.



### Anahtar rotasyonu

Transfer sonrası taahhüt değişir → eski anahtar tekrar kullanılamaz ("Public key mismatch").



### Mesaj bağlama

İmza belirli alıcı + miktar + nonce'a bağlıdır → başka bir transfere kopyalanamaz.



**test\_ReplayAfterRotationFails** bunu kanıtlar: rotasyondan sonra eski anahtarla yapılan transfer "Public key mismatch" ile reddedilir.

## ■ Test: Foundry ile Doğrulama

| Test                             | Ne doğrular                     | Sonuç |
|----------------------------------|---------------------------------|-------|
| test_Register                    | Kayıt + başlangıç bakiyesi      | ✓     |
| test_TransferValid               | Geçerli imzayla transfer        | ✓     |
| test_TransferInvalidSignature    | Bozuk imza reddedilir (revert)  | ✓     |
| test_TransferInsufficientBalance | Yetersiz bakiye reddedilir      | ✓     |
| test_ReplayAfterRotationFails    | Eski anahtar reddedilir         | ✓     |
| test_Custom                      | Parametrelili senaryo (env ile) | ✓     |

6 / 6

TÜM TESTLER GEÇTİ

forge test

birim testleri + revert  
senaryoları doğrular

Hem başarı yolu (geçerli transfer) hem de hata yolları (bozuk imza, yetersiz bakiye, replay) test edilir. test\_Custom, eğitmenin kendi girdileriyle çalıştırabileceği parametrelili bir senaryodur.

## ■ Eksiler



### Büyük boyut

İmza  $\sim 8$  KB ( $256 \times 32$  byte), açık anahtar  $\sim 16$  KB.



### Tek kullanımlık

Her transfer anahtar rotasyonu gerektirir; durum yönetimi artar.



### Yüksek gas

$256 \text{ sha256} + \text{büyük calldata} \rightarrow$  işlem maliyeti yüksektir.



### Ölçeklenme

Pratikte XMSS/SPHINCS+ veya kafes-tabanlı Dilithium tercih edilir.

## ■ Özet



ECDSA kuantum bilgisayarda kırılır; hash tabanlı Lamport post-kuantum güvenlidir.



Token transferleri Lamport imzasıyla yetkilendirilir; anahtar her transferde döner.



İki hash: SHA-256 (Lamport şeması) + keccak256 (kompakt zincir taahhüdü).



Nonce + rotasyon + mesaj bağlama ile replay ve sahtecilik engellenir.